

P2 - Analytics Engineering Platform

dbt + AIRFLOW + DOCKER | PROJECT BOOK

Business story in beginner language

A subscription company has raw billing/customer data, but different analysts calculate revenue differently. Your job is to turn raw data into one tested, documented analytical model and make the build repeatable. A wrong uniqueness/relationship result must stop trusted publish.

What you are building

A containerized local analytics engineering platform using dbt models/tests, Airflow orchestration, safe incremental logic, late-data/backfill behavior, lineage and runbook evidence.

What this project is really testing

- Model grain and dimensional thinking.
- dbt source/ref lineage and model layers.
- Generic/custom tests as quality gates.
- Incremental merge/rerun/late-data reasoning.
- Airflow dependency/retry/partial-failure behavior.
- Docker configuration and reproducible setup.

Important: teaching demo vs your protected project

The mini example below teaches the pattern using different toy data. It is NOT the final solution for your supplied project. After you understand the pattern, close the example and apply the reasoning to the project assets yourself.

Tiny teacher demonstration

Toy case: raw billing to tested revenue mart

Suppose raw_billing has one row per invoice, raw_customers one row per customer. You first stage/rename/cast, then create a revenue fact. The fact grain must stay one row per invoice unless you intentionally change it.

```
raw_billing -> stg_billing -> int_billing_enriched -> fct_revenue
raw_customers -> stg_customers -----^
Critical tests:
- invoice_id: not_null + unique
- customer_id relationship to customer model
- revenue >= 0
- downstream publish runs only after tests pass
```

dbt ref()/source() creates explicit dependency lineage. Airflow should orchestrate the steps; it should not hide a failed quality test and still mark "publish" successful.

Incremental in plain English

Incremental means you avoid rebuilding all history on every normal run. But you must state unique key, new/changed-row filter, late-data policy and how to reprocess history when a correction arrives.

```
normal daily run: process rows where updated_at > watermark
late correction: bounded backfill date range
merge key: invoice_id
quality gate: tests must pass before publish
```

STOP - explain before continuing

If a duplicate invoice_id arrives and the uniqueness test fails, why should the Gold/publish step not continue?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Your project build order

Phase 1 - Define analytical grains and business questions

First understand

dbt cannot rescue a model whose row meaning is unclear.

Then do it in your project

1. Profile billing/customers/late_billing.
2. Write fact/dimension grains and keys.
3. Write 3-5 business questions the marts should answer.

Evidence gate - prove before move-on

- fct_revenue grain explicit
- customer dimension key explicit
- business questions map to outputs

STOP - explain before continuing

Before Phase 2: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 2 - Set up sources and staging

First understand

Staging models should clean names/types close to sources without sneaking in business aggregation.

Then do it in your project

1. Define dbt sources.
2. Build stg_billing/stg_customers with ref/source dependencies.
3. Cast/rename/standardize.

Evidence gate - prove before move-on

- source lineage visible
- staging grain preserved
- staging SQL nontrivial and explainable

STOP - explain before continuing

Before Phase 3: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 3 - Build intermediate and marts

First understand

Intermediate models hold reusable business transformations; marts expose trusted analytical outputs.

Then do it in your project

1. Create enrichment joins at compatible grain.
2. Build fact/dimension outputs.
3. Check joins do not multiply revenue.

Evidence gate - prove before move-on

- row grain stated per model
- revenue reconciles to accepted source
- relationships intentional

STOP - explain before continuing

Before Phase 4: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 4 - Add generic + custom tests

First understand

Tests are publish gates, not decorative YAML.

Then do it in your project

1. not_null/unique on identifiers.
2. relationships on foreign keys.
3. accepted/range/business tests.
4. custom positive-revenue rule.

Evidence gate - prove before move-on

- healthy data passes
- supplied duplicate/broken relationship fails expected test
- failure names root rule

STOP - explain before continuing

Before Phase 5: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 5 - Design incremental model

First understand

Incremental needs unique key, filter/watermark and late/backfill behavior.

Then do it in your project

1. Choose incremental fact/model.
2. Define unique_key/merge behavior.
3. Define normal new-row filter.
4. Write late_billing reprocess rule.

Evidence gate - prove before move-on

- rerun does not duplicate logical invoices
- late row becomes visible after bounded backfill
- full refresh/backfill path documented

STOP - explain before continuing

Before Phase 6: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 6 - Orchestrate with Airflow

First understand

Airflow controls dependency/order/retries. Quality failure must block publish.

Then do it in your project

1. Build DAG load -> dbt build/test -> publish.
2. Set retries only where safe.
3. Ensure publish depends on successful quality.

Evidence gate - prove before move-on

- task dependency graph explainable
- failure path blocks publish
- retry does not create duplicates

STOP - explain before continuing

Before Phase 7: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 7 - Package reproducible local services

First understand

Docker Compose should describe services/config, not embed credentials.

Then do it in your project

1. Review services/volumes/env references.
2. Use .env.example or variables.
3. Document startup/health/run order.

Evidence gate - prove before move-on

- no plaintext secret
- clean setup path documented
- state persistence behavior understood

STOP - explain before continuing

Before Phase 8: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 8 - Trigger dbt + Airflow incidents

First understand

Repair both data/model quality and orchestration partial failure.

Then do it in your project

1. Save uniqueness/relationship failure.
2. Fix root data/model issue.
3. Save Airflow partial failure evidence.
4. Rerun only safe tasks then full checks.

Evidence gate - prove before move-on

- failed Gold not published
- repair evidence saved
- late/backfill still correct

STOP - explain before continuing

Before Phase 9: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 9 - Finish lineage/runbook/defense

First understand

Make the project understandable without opening every SQL file.

Then do it in your project

1. Complete lineage.md.
2. Complete README/runbook.
3. Run checker/static tests available.
4. Practice defense.

Evidence gate - prove before move-on

- lineage source->staging->mart clear
- runbook covers late/backfill/recovery
- limitations honest

STOP - explain before continuing

Before Phase complete: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Expected control evidence

Control	Expected reasoning
staging	preserves source row meaning after cleanup
fact grain	explicit invoice/billing grain
generic tests	unique + not_null + relationships
custom test	positive/business revenue rule
incremental	unique key + rerun + late-data/backfill
orchestration	publish blocked after failed critical test

Deliberate failures you must practise

Duplicate business key

Supplied billing duplicate should break uniqueness or reveal unsafe merge.

Save first failing evidence before repair.

- save dbt test failure
- repair data/model rule
- retest + reconcile

Broken relationship

A billing customer key has no valid customer.

Save first failing evidence before repair.

- relationship test fails
- do not silently publish orphan
- decide reject/repair/source-owner action

Airflow partial failure

Upstream work succeeded but downstream task failed.

Save first failing evidence before repair.

- explain task states
- rerun only safe task/path
- prove publish not falsely fresh

Technical defense

What is the exact grain of fct_revenue?

Your answer: _____

Evidence you would point to: _____

Why did you separate staging/intermediate/marts?

Your answer: _____

Evidence you would point to: _____

What causes the incremental model to include a row?

Your answer: _____

Evidence you would point to: _____

How does late_billing get repaired without rebuilding everything blindly?

Your answer: _____

Evidence you would point to: _____

What happens to publish when dbt tests fail?

Your answer: _____

Evidence you would point to: _____

What does Docker solve here and what does it NOT solve?

Your answer: _____

Evidence you would point to: _____

Honest limitations

A local Compose project is not the same as production orchestration/warehouse deployment. State missing HA, secret manager, centralized observability and production data-volume evidence rather than hiding them.

Evidence gate - prove before move-on

- README explains what is local/simulated.
- No claim of production scale without evidence.
- You can say what you would change at higher volume/availability/security requirements.